

TRƯỜNG ĐẠI HỌC FPT
KHOA CÔNG NGHỆ THÔNG TIN
(FPT University)

BÁO CÁO CUỐI KỲ

**Môn học: AI trong sản xuất
DevOps, DataOps, MLOps**

Mã môn: DDM501

Đề tài:

Xây dựng hệ thống MLOps end-to-end

Phân loại chất lượng rượu vang với giám sát, phát hiện drift
và tự động tái huấn luyện mô hình

Sinh viên thực hiện	Mã sinh viên
Phạm Minh Hoàng	25MS13285
Nguyễn Công Trọng	25MS13297
Ngô Minh Khôi	25MSA13236

Năm học: 2025 – 2026

Học kỳ: 1 – 2026

Hà Nội, tháng 10 năm 2026

Lời cam đoan

Chúng tôi xin cam đoan rằng báo cáo cuối kỳ này là công trình nghiên cứu và thực nghiệm của nhóm, được thực hiện dưới sự hướng dẫn của giảng viên môn học. Các số liệu, kết quả được trình bày trong báo cáo là trung thực và chưa từng được công bố ở bất kỳ công trình nào khác. Mọi tham khảo từ tài liệu nước ngoài đều được trích dẫn đầy đủ.

Hà Nội, tháng 10 năm 2026

Nhóm sinh viên thực hiện

Phạm Minh Hoàng Nguyễn Công Trọng Ngô Minh Khôi

Tóm tắt

Báo cáo này trình bày quá trình thiết kế và triển khai một hệ thống MLOps hoàn chỉnh (end-to-end) cho bài toán phân loại chất lượng rượu vang (*Wine Quality Classification*) sử dụng bộ dữ liệu `sklearn.datasets.load_wine()`. Hệ thống bao gồm toàn bộ vòng đời của một mô hình học máy: từ thu thập dữ liệu, tiền xử lý, huấn luyện, phục vụ (serving), giám sát (monitoring), phát hiện data drift đến tự động tái huấn luyện.

Kiến trúc hệ thống được xây dựng với 8 dịch vụ Docker: **FastAPI** cho REST API serving, **MLflow** cho theo dõi thí nghiệm và quản lý mô hình, **Apache Airflow** cho orchestration pipeline tự động, **Evidently AI** cho phát hiện data drift, **Prometheus** và **Grafana** cho thu thập và trực quan hóa metrics, cùng **PostgreSQL** làm cơ sở dữ liệu nền.

Kết quả thực nghiệm cho thấy hệ thống đạt độ chính xác 100% trên tập kiểm tra tĩnh, vượt qua 74 test cases với độ bao phủ code 90% ($\geq 80\%$ theo yêu cầu), và Airflow DAG gồm 5 tasks hoàn thành thành công khi nhận tín hiệu drift.

Từ khóa: MLOps, data drift, model monitoring, Docker, FastAPI, MLflow, Apache Airflow, Evidently AI, Prometheus, Grafana, Wine Classification.

Mục lục

Danh sách hình vẽ

Danh sách bảng

Chương 1

Giới thiệu

1.1 Bối cảnh và động lực

Trong kỷ nguyên số hóa, các mô hình học máy (Machine Learning – ML) đang được triển khai ngày càng rộng rãi trong các hệ thống sản xuất thực tế. Tuy nhiên, việc đưa một mô hình từ giai đoạn nghiên cứu vào môi trường production và duy trì hiệu năng theo thời gian là một thách thức lớn. Đây chính là vấn đề mà MLOps (Machine Learning Operations) được sinh ra để giải quyết.

MLOps kết hợp các thực hành tốt nhất từ DevOps, DataOps và kỹ thuật học máy nhằm tạo ra một quy trình tự động, có thể tái lập và liên tục cải thiện. Theo báo cáo của Gartner (2023), hơn 80% các dự án ML không bao giờ đến được giai đoạn production do thiếu hạ tầng vận hành phù hợp. MLOps chính là cầu nối giúp thu hẹp khoảng cách này.

1.2 Mục tiêu đề tài

Đề tài đặt ra các mục tiêu cụ thể sau:

1. **Xây dựng pipeline ML hoàn chỉnh** từ ingestion đến serving với kiến trúc microservices dựa trên Docker.
2. **Triển khai MLflow** để theo dõi thí nghiệm, versioning mô hình và quản lý lifecycle.
3. **Phát triển REST API** với FastAPI để phục vụ dự đoán theo thời gian thực.
4. **Thiết lập hệ thống monitoring** với Prometheus + Grafana để theo dõi metrics về hiệu năng và sức khỏe hệ thống.
5. **Phát hiện data drift tự động** bằng Evidently AI và kích hoạt tái huấn luyện thông qua Airflow.
6. **Đảm bảo chất lượng code** với bộ test cases đạt coverage $\geq 80\%$.
7. **Tích hợp CI/CD** với GitHub Actions để tự động hóa kiểm thử và triển khai.

1.3 Phạm vi thực hiện

Báo cáo tập trung vào các nội dung sau:

- **Bài toán:** Phân loại chất lượng rượu vang (3 lớp: class 0, 1, 2) dựa trên 13 thuộc tính hóa học.
- **Dataset:** `sklearn.datasets.load_wine()` – 178 mẫu, 13 features.

- **Mô hình:** Random Forest Classifier (sklearn).
- **Môi trường:** Docker Desktop trên Windows 11, Python 3.11.
- **Thời gian thực hiện:** Tháng 9–10/2026.

1.4 Cấu trúc báo cáo

Báo cáo được tổ chức thành 7 chương:

- **Chương 1:** Giới thiệu tổng quan.
- **Chương 2:** Cơ sở lý thuyết về MLOps.
- **Chương 3:** Kiến trúc và thiết kế hệ thống.
- **Chương 4:** Mô tả dữ liệu và mô hình ML.
- **Chương 5:** Quy trình triển khai và thực nghiệm.
- **Chương 6:** Kết quả đạt được.
- **Chương 7:** Kết luận và hướng phát triển.

Chương 2

Cơ sở lý thuyết

2.1 MLOps là gì?

MLOps (Machine Learning Operations) là tập hợp các thực hành, công cụ và quy trình nhằm chuẩn hóa và tự động hóa vòng đời của mô hình học máy trong môi trường sản xuất. MLOps kế thừa các nguyên tắc từ DevOps và áp dụng vào lĩnh vực học máy.

**Data → Model Training → Evaluation → Deployment → Monitoring
→ Re-training**

(Vòng lặp liên tục – Continuous ML)

Hình 2.1: Vòng đời của mô hình học máy trong MLOps

2.1.1 Các thành phần cốt lõi của MLOps

Experiment Tracking: Ghi lại các thí nghiệm, siêu tham số (hyperparameters), metrics và artifacts của mỗi lần huấn luyện.

Model Registry: Kho lưu trữ và quản lý phiên bản mô hình với các trạng thái (Staging, Production, Archived).

Pipeline Orchestration: Tự động hóa quy trình từ ingestion đến deployment bằng các công cụ như Airflow, Prefect, Kubeflow.

Model Serving: Phục vụ dự đoán qua REST API, gRPC hoặc batch inference.

Monitoring: Theo dõi hiệu năng mô hình, data drift và infrastructure health.

Continuous Training: Tự động tái huấn luyện khi phát hiện model degradation hoặc data drift.

2.2 Data Drift và Model Drift

Data Drift (hay *Dataset Shift*) là hiện tượng phân phối của dữ liệu đầu vào thay đổi theo thời gian so với dữ liệu huấn luyện. Đây là một trong những nguyên nhân chính dẫn đến suy giảm hiệu năng mô hình trong production.

Có ba loại drift phổ biến:

- **Covariate Shift:** Phân phối đầu vào $P(X)$ thay đổi nhưng $P(Y|X)$ giữ nguyên.
- **Label Shift:** Phân phối nhãn $P(Y)$ thay đổi.

- **Concept Drift:** Mỗi quan hệ $P(Y|X)$ thay đổi theo thời gian.

Trong đề tài này, chúng tôi giả lập **Covariate Shift** bằng cách thêm nhiễu Gaussian vào dữ liệu input ($\mu = 0, \sigma = 2.0$) để kích hoạt quy trình phát hiện drift.

2.3 Các công cụ sử dụng

2.3.1 MLflow

MLflow là một nền tảng mã nguồn mở do Databricks phát triển, cung cấp 4 thành phần chính: *Tracking*, *Projects*, *Models* và *Registry*. Trong đề tài, chúng tôi sử dụng *Tracking* và *Registry*.

2.3.2 Apache Airflow

Apache Airflow là công cụ workflow orchestration mã nguồn mở, cho phép định nghĩa, lên lịch và giám sát các pipeline dưới dạng DAG (Directed Acyclic Graph). Mỗi node trong DAG là một Task.

2.3.3 FastAPI

FastAPI là một web framework Python hiện đại, hiệu năng cao, dựa trên ASGI. FastAPI tự động sinh tài liệu Swagger UI và hỗ trợ kiểm tra kiểu dữ liệu với Pydantic.

2.3.4 Evidently AI

Evidently AI là thư viện Python chuyên biệt để đánh giá, kiểm tra và giám sát mô hình ML. Nó cung cấp các báo cáo về chất lượng dữ liệu, data drift và model performance.

2.3.5 Prometheus & Grafana

Prometheus là hệ thống time-series database dùng để thu thập metrics qua HTTP scraping. Grafana là công cụ trực quan hóa metrics, kết nối với Prometheus để xây dựng dashboards.

2.3.6 Docker & Docker Compose

Docker cho phép đóng gói ứng dụng cùng dependencies vào container, đảm bảo tính nhất quán giữa các môi trường. Docker Compose quản lý multi-container applications.

Chương 3

Kiến trúc và Thiết kế Hệ thống

3.1 Tổng quan kiến trúc

Hệ thống được thiết kế theo kiến trúc microservices, với 8 dịch vụ độc lập giao tiếp qua Docker network monitoring. Tất cả dịch vụ được định nghĩa trong file `docker-compose.yml` và chia sẻ volume chung `/mlflow` để truy cập artifacts.

Service	Port	Vai trò
postgres	5432	Database (Airflow metadata, MLflow)
mlflow	5000	Experiment tracking & Model registry
api (FastAPI)	8000	REST API serving dự đoán
evidently	8001	Data drift detection service
prometheus	9090	Metrics collection
grafana	3000	Metrics visualization (dashboard)
airflow-webserver	8080	Pipeline orchestration UI
airflow-scheduler	--	DAG scheduler (background)

Hình 3.1: Danh sách các dịch vụ trong hệ thống

3.2 Luồng dữ liệu tổng thể

Luồng hoạt động chính của hệ thống được mô tả như sau:

- Training phase:** Script `training.py` load dữ liệu wine, huấn luyện `RandomForestClassifier`, log metrics lên MLflow và đăng ký mô hình vào Production.
- Serving phase:** FastAPI API load mô hình từ MLflow Registry và cung cấp endpoint `POST /predict` cho client.
- Monitoring phase:** Mỗi request được ghi lại metrics (latency, error rate, prediction distribution). Prometheus scrape metrics theo chu kỳ.
- Drift Detection:** Script gửi 100 request bình thường, sau đó 250 request với dữ liệu bị nhiễu. Evidently AI phân tích và phát hiện drift khi tỷ lệ drift $\geq 30\%$.
- Auto-Retraining:** Khi drift được phát hiện, hệ thống kích hoạt Airflow DAG `ml_pipeline` thông qua REST API.
- New Model:** Airflow DAG chạy 5 tasks: `ingest` $\rightarrow$ `clean` $\rightarrow$ `feature engineer` $\rightarrow$ `train` $\rightarrow$ `evaluate & register phiên bản mới vào MLflow`.

3.3 Cấu trúc thư mục dự án

Listing 3.1: Cấu trúc thư mục dự án ml-monitoring

```

1 ml-monitoring/
2 |-- airflow/
3 |   |-- dags/
4 |       |-- ml_pipeline_dag.py    # Airflow DAG 5 tasks
5 |-- api/
6 |   |-- Dockerfile
7 |   |-- main.py                  # FastAPI application
8 |   |-- requirements.txt
9 |-- evidently/
10 |   |-- Dockerfile
11 |   |-- main.py                  # Evidently drift service
12 |-- mlflow/
13 |   |-- Dockerfile              # MLflow server
14 |-- monitoring/
15 |   |-- grafana/                # Grafana dashboards
16 |   |-- prometheus/
17 |       |-- prometheus.yml      # Scrape config
18 |-- scripts/
19 |   |-- training.py              # Initial training script
20 |   |-- simulate_drift_and_retrain.py
21 |-- src/                        # Core ML modules
22 |   |-- data_pipeline.py
23 |   |-- feature_engineering.py
24 |   |-- train.py
25 |   |-- evaluate.py
26 |   |-- explainability.py
27 |-- tests/                      # Unit & integration tests
28 |-- .github/workflows/
29 |   |-- ci-cd.yml               # GitHub Actions CI/CD
30 |-- docker-compose.yml

```

3.4 Thiết kế Airflow DAG

DAG ml_pipeline được định nghĩa với 5 tasks thực thi tuần tự:

```

data_ingestion → data_cleaning → feature_engineering →
model_training → model_evaluation_and_registration

```

Hình 3.2: Airflow DAG – 5 tasks ML pipeline

data_ingestion: Load dữ liệu wine từ sklearn, lưu vào /tmp/raw_data.pkl.

data_cleaning: Kiểm tra và xử lý missing values, outliers.

feature_engineering: Chuẩn hóa features bằng StandardScaler, split train/test.

model_training: Huấn luyện RandomForestClassifier với cross-validation, log metrics MLflow.

model_evaluation_and_registration: Đánh giá trên test set, nếu accuracy $\geq 85\%$ thì đăng ký vào Production.

3.5 Kiến trúc CI/CD

GitHub Actions workflow được cấu hình với 3 jobs:

1. **test:** Chạy pytest với coverage check ($\geq 80\%$).
2. **lint:** Kiểm tra code style với flake8.
3. **build:** Build Docker images (chạy trên main branch).

Chương 4

Mô tả Dữ liệu và Mô hình Học Máy

4.1 Dataset: Wine Quality

Bộ dữ liệu được sử dụng là `sklearn.datasets.load_wine()`, một bộ dữ liệu kinh điển trong học máy. Đây là kết quả phân tích hóa học của 178 mẫu rượu vang từ ba giống nho trồng ở vùng Piedmont, Ý.

Bảng 4.1: Thông tin tổng quan về dataset Wine

Thuộc tính	Giá trị
Số mẫu (samples)	178
Số features	13
Số lớp (classes)	3 (class_0: 59, class_1: 71, class_2: 48)
Missing values	Không có
Kiểu dữ liệu	Float64

4.1.1 Mô tả 13 features

Bảng 4.2: Mô tả các features trong dataset Wine

	STT	Feature	Mô tả
1		alcohol	Nồng độ cồn (%)
2		malic_acid	Axit malic (g/L)
3		ash	Tro (g/L)
4		alcalinity_of_ash	Độ kiềm của tro
5		magnesium	Magie (mg/L)
6		total_phenols	Tổng phenol
7		flavanoids	Flavanoid
8		nonflavanoid_phenols	Phenol không phải flavanoid
9		proanthocyanins	Proanthocyanin
10		color_intensity	Cường độ màu sắc
11		hue	Sắc độ
12		od280/od315_of_diluted_wines	Tỷ lệ OD280/OD315 của rượu pha loãng
13		proline	Proline (mg/L)

4.2 Mô hình học máy

4.2.1 Lựa chọn mô hình

Chúng tôi sử dụng **Random Forest Classifier** từ thư viện scikit-learn vì các lý do:

- Xử lý tốt dữ liệu đa chiều mà không cần feature selection thủ công.
- Ít bị overfitting nhờ cơ chế ensemble averaging.
- Cung cấp feature importance giúp giải thích mô hình (explainability).
- Không yêu cầu chuẩn hóa dữ liệu đầu vào (robust với scale khác nhau).

4.2.2 Hyperparameters

Bảng 4.3: Hyperparameters của RandomForestClassifier

Tham số	Giá trị	Giải thích
n_estimators	100	Số cây quyết định
max_depth	None	Không giới hạn độ sâu
random_state	42	Seed cố định để tái lập
n_jobs	-1	Dùng tất cả CPU cores
test_size	0.2 (80/20)	Tỷ lệ train/test split
cv_folds	5	5-fold cross-validation

4.2.3 Quy trình tiền xử lý dữ liệu

1. **Load data:** `sklearn.datasets.load_wine()` trả về (X, y) với $X \in \mathbb{R}^{178 \times 13}$, $y \in \{0, 1, 2\}^{178}$.
2. **Train-test split:** Phân chia 80% train / 20% test, stratified theo nhãn.
3. **Feature scaling:** `StandardScaler` chuẩn hóa mỗi feature về $\mu = 0$, $\sigma = 1$.
4. **Cross-validation:** 5-fold CV trên tập train để ước lượng hiệu năng khái quát.

4.2.4 Explainability – Giải thích mô hình

Module `src/explainability.py` cung cấp:

- **Feature Importance:** Xếp hạng 13 features theo mức độ đóng góp.
- **Prediction Explanation:** Giải thích từng dự đoán cụ thể.
- **Model Summary Report:** Báo cáo tổng hợp về mô hình.

Chương 5

Quy trình Triển khai và Thực nghiệm

5.1 Môi trường triển khai

Bảng 5.1: Thông tin môi trường triển khai

Thành phần	Chi tiết
Hệ điều hành	Windows 11 Pro
Docker Desktop	Docker Engine 27.x
Python	3.11
scikit-learn	$\geq 1.5.0$
numpy	$\geq 2.0.0$
MLflow	2.x (build from Dockerfile)
Airflow image	ddm501-t03-airflow:2.8.4
Prometheus	v2.54.1
Grafana	11.2.0
PostgreSQL	15-alpine

5.2 Các bước triển khai

5.2.1 Bước 1: Khởi động hạ tầng

Listing 5.1: Khởi động toàn bộ 8 services bằng Docker Compose

```
1 # Di chuyển vào thư mục dự án
2 cd ml-monitoring
3
4 # Khi động tất cả services để ch độ daemon
5 docker compose up -d
6
7 # Kiểm tra trạng thái
8 docker compose ps
```

Sau khi chạy lệnh trên, 8 containers sẽ được khởi động và sẵn sàng sau khoảng 30-60 giây.

5.2.2 Bước 2: Huấn luyện mô hình ban đầu

Listing 5.2: Huấn luyện mô hình lần đầu

```
1 python scripts/training.py
```

Script `training.py` thực hiện:

- Load dataset wine từ sklearn.
- Huấn luyện RandomForestClassifier.
- Log metrics (accuracy, precision, recall, F1) lên MLflow.
- Đăng ký mô hình `wine_quality_model` Version 1 vào stage **Production**.

5.2.3 Bước 3: Demo drift và tái huấn luyện tự động

Listing 5.3: Chạy demo giả lập data drift

```
1 python scripts/simulate_drift_and_retrain.py
```

Kịch bản mô phỏng:

1. Gửi **100 request bình thường** đến API POST `/predict` với dữ liệu gốc.
2. Gửi **250 request với dữ liệu bị nhiễu** (Gaussian noise: $\mu = 0, \sigma = 2.0$).
3. Evidently AI phát hiện drift ($\geq 30\%$ features bị drift).
4. Kích hoạt Airflow DAG `ml_pipeline` qua REST API.
5. DAG chạy tự động 5 tasks và đăng ký model phiên bản mới.

5.2.4 Bước 4: Giám sát và quan sát kết quả

Sau khi hệ thống chạy, người dùng có thể truy cập các giao diện:

Bảng 5.2: Danh sách URL truy cập các thành phần hệ thống

Dịch vụ	URL	Thông tin đăng nhập
MLflow Tracking	http://localhost:5000	Không cần
Airflow DAGs	http://localhost:8080	admin / admin
FastAPI Swagger	http://localhost:8000/docs	Không cần
Grafana Dashboard	http://localhost:3000	admin / admin
Prometheus Targets	http://localhost:9090/targets	Không cần
Evidently Service	http://localhost:8001	Không cần

5.3 Kiểm thử (Testing)

5.3.1 Cấu trúc bộ test

Bảng 5.3: Danh sách test files và số lượng test cases

File test	Module được test	Số tests
test_data.py	src/data_pipeline.py	~12
test_train.py	src/train.py	~10
test_evaluate.py	src/evaluate.py	~14
test_explainability.py	src/explainability.py	~10
test_api.py	api/main.py (FastAPI)	~15
test_model.py	Integration tests	~13
Tổng cộng		74 tests

5.3.2 Chạy kiểm thử

Listing 5.4: Chạy bộ test với coverage report

```

1  # Cài đặt dependencies
2  pip install pytest pytest-cov
3
4  # Chạy toàn bộ test với coverage
5  pytest tests/ --cov=src --cov=api --cov-report=term-missing -v
6
7  # Kết quả: 74 tests PASSED -- coverage: 90%
```

5.4 Xử lý các thách thức kỹ thuật

Trong quá trình triển khai, nhóm đã gặp và giải quyết một số thách thức kỹ thuật đáng chú ý:

- MinIO Docker image bị ngừng hỗ trợ (October 2025):** Ban đầu hệ thống được thiết kế dùng MinIO làm artifact store. Tuy nhiên, image minio/minio không còn pull được từ Docker Hub. Giải pháp: thay thế hoàn toàn bằng local file storage (file:///mlflow/artifacts), mount volume C:/mlflow:/mlflow vào tất cả containers cần truy cập artifact.
- NumPy version mismatch:** Container API dùng NumPy 1.24.3 không đọc được model pickle được lưu bởi NumPy 2.x (lỗi No module named 'numpy._core'). Giải pháp: nâng lên numpy>=2.0.0 trong api/requirements.txt.
- UnicodeEncodeError trên Windows:** Scripts sử dụng emoji trong print gây lỗi UnicodeEncodeError với encoding mặc định của Windows terminal (cp1252). Giải pháp: thêm sys.stdout.reconfigure() đầu mỗi script.
- Airflow Permission Denied trên MLflow artifacts:** Task model_training lỗi Permission denied: '/mlflow' do airflow container không được mount volume. Giải pháp: thêm C:/mlflow:/mlflow vào cả airflow-webserver và airflow-scheduler.

Chương 6

Kết quả Đạt được

6.1 Kết quả huấn luyện mô hình

Kết quả

Model: wine_quality_model – Version 1

Stage: Production

Dataset: sklearn Wine (178 mẫu, 80% train / 20% test = 142/36 mẫu)

Accuracy (test set)	100%
Precision (macro avg)	1.000
Recall (macro avg)	1.000
F1-score (macro avg)	1.000
Cross-validation (5-fold)	97.9% $\pm$ 2.8%

Ghi chú

Độ chính xác 100% trên tập kiểm tra không phải do overfitting. Dataset Wine là bộ dữ liệu nhỏ nhưng có sự phân biệt rõ ràng giữa các lớp. Random Forest với 100 cây đủ để học đầy đủ các ranh giới quyết định. Cross-validation 5-fold cho 97.9% xác nhận mô hình có khả năng khái quát tốt.

6.2 Kết quả kiểm thử tự động

Kết quả

Tổng số test cases: 74

Kết quả: 74 PASSED, 0 FAILED, 0 ERROR

Code Coverage: 90% (yêu cầu $\geq$ 80%)

Thời gian chạy: ~45 giây

6.3 Kết quả Airflow DAG

Sau khi script drift detection phát hiện data drift, Airflow DAG được kích hoạt và hoàn thành thành công:

Bảng 6.1: Kết quả thực thi Airflow DAG

Task	Trạng thái	Duration	Ghi chú
data_ingestion	SUCCESS	~2s	Load wine dataset
data_cleaning	SUCCESS	~1s	No missing values
feature_engineering	SUCCESS	~1s	StandardScaler fit
model_training	SUCCESS	~8s	RF 100 estimators
model_evaluation_and_registration	SUCCESS	~3s	Đăng ký Production
Tổng	5/5 SUCCESS	~15s	

6.4 Kết quả giám sát hệ thống

6.4.1 Prometheus Metrics

Prometheus thu thập các metrics sau từ FastAPI:

- `http_requests_total`: Tổng số requests theo endpoint và status code.
- `http_request_duration_seconds`: Phân phối latency theo histogram.
- `model_predictions_total`: Số dự đoán theo class.
- `model_prediction_latency`: Thời gian inference.

6.4.2 Grafana Dashboard

Grafana hiển thị 4 panels chính:

1. **Request Rate**: Số request/giây theo thời gian thực.
2. **Response Time**: Phân phối latency P50/P95/P99.
3. **Prediction Distribution**: Tỷ lệ dự đoán class 0/1/2.
4. **Error Rate**: Tỷ lệ requests lỗi (4xx, 5xx).

6.5 Kết quả CI/CD

GitHub Actions workflow được cấu hình tại `.github/workflows/ci-cd.yml`:

- **Trigger**: Push lên branch main hoặc develop, hoặc mở Pull Request.
- **Jobs**: Test (với coverage check) → Lint → Build.
- **Status**: Tất cả jobs PASS khi code trên main branch.

6.6 So sánh với yêu cầu đề tài

Bảng 6.2: So sánh kết quả với yêu cầu đề tài

Yêu cầu	Mục tiêu	Kết quả
Model accuracy	$\geq 85\%$	100%
Code coverage	$\geq 80\%$	90%
Số test cases	≥ 10	74
DAG tasks	≥ 3	5
Drift detection	Có	Có (Evidently AI)
Auto-retraining	Có	Có (Airflow)
REST API serving	Có	Có (FastAPI)
Monitoring dashboard	Có	Có (Grafana)
CI/CD pipeline	Có	Có (GitHub Actions)

Chương 7

Kết luận và Hướng Phát triển

7.1 Kết luận

Đề tài đã xây dựng thành công một hệ thống MLOps hoàn chỉnh, đáp ứng đầy đủ yêu cầu của môn học. Cụ thể:

1. **Pipeline ML tự động:** Toàn bộ vòng đời mô hình từ training đến monitoring được tự động hóa, không cần can thiệp thủ công sau khi hệ thống khởi động.
2. **Kiến trúc microservices:** 8 dịch vụ Docker chạy độc lập, dễ dàng scale và thay thế từng thành phần mà không ảnh hưởng đến toàn hệ thống.
3. **Vòng lặp khép kín:** Hệ thống tự động phát hiện data drift và kích hoạt tái huấn luyện, đảm bảo mô hình luôn cập nhật với phân phối dữ liệu mới.
4. **Chất lượng code:** 74 test cases với 90% coverage, vượt qua yêu cầu tối thiểu 80%.
5. **Khả năng quan sát (Observability):** Prometheus + Grafana cung cấp visibility đầy đủ về hiệu năng hệ thống và mô hình.

Qua quá trình thực hiện, nhóm đã tích lũy kinh nghiệm quý báu về:

- Cách thiết kế và debug hệ thống multi-container phức tạp.
- Xử lý các vấn đề tương thích phiên bản trong stack Python/numpy/sklearn.
- Vận hành Apache Airflow cho ML pipeline orchestration.
- Áp dụng các kỹ thuật phát hiện data drift trong thực tế.

7.2 Hướng phát triển

Hệ thống hiện tại có thể được phát triển theo các hướng sau:

1. **Triển khai lên Kubernetes (K8s):** Sử dụng Helm charts để deploy lên cloud (GCP, AWS, Azure) với khả năng auto-scaling.
2. **Tích hợp Feature Store:** Sử dụng Feast hoặc Hopsworks để quản lý features tập trung, chia sẻ giữa nhiều mô hình.
3. **A/B Testing:** Triển khai nhiều phiên bản mô hình song song với traffic splitting để so sánh hiệu năng trong production.
4. **Hyperparameter Optimization tự động:** Tích hợp Optuna hoặc Ray Tune vào DAG để tự động tìm kiếm hyperparameters tối ưu khi retrain.

5. **Data versioning:** Sử dụng DVC (Data Version Control) để versioning dữ liệu song song với model versioning.
6. **Mô hình phức tạp hơn:** Áp dụng với deep learning (TensorFlow/PyTorch) và các bài toán phức tạp hơn như NLP, time-series forecasting.
7. **Security:** Thêm authentication cho các API endpoints, mã hóa secrets với HashiCorp Vault, và network policies cho Docker.

Tài liệu tham khảo

- [1] Sculley, D., et al. (2015). *Hidden Technical Debt in Machine Learning Systems*. NIPS 2015.
- [2] Zaharia, M., et al. (2018). *Accelerating the Machine Learning Lifecycle with MLflow*. VLDB 2018.
- [3] Apache Software Foundation. (2024). *Apache Airflow Documentation*. <https://airflow.apache.org/docs/>
- [4] Tikhonov, M., et al. (2022). *Evidently: An Open-Source Framework for Data and ML Monitoring*. <https://docs.evidentlyai.com/>
- [5] FastAPI Documentation. (2024). <https://fastapi.tiangolo.com/>
- [6] Prometheus Documentation. (2024). *Prometheus Monitoring System & Time Series Database*. <https://prometheus.io/docs/>
- [7] Grafana Labs. (2024). *Grafana Documentation*. <https://grafana.com/docs/>
- [8] Scikit-learn Developers. (2024). *sklearn.datasets.load_wine*. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_wine.html
- [9] Breck, E., et al. (2017). *The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction*. IEEE Big Data 2017.
- [10] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- [11] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
- [12] Alla, S., & Adari, S. K. (2021). *Beginning MLOps with MLflow*. Apress.

Chương A

Hướng dẫn cài đặt và chạy

A.1 Yêu cầu hệ thống

- Docker Desktop (đã bật và đang chạy)
- Python 3.11+
- Git
- RAM: tối thiểu 8GB (khuyến nghị 16GB)
- Disk: tối thiểu 5GB trống

A.2 Các bước cài đặt từ đầu

Listing A.1: Cài đặt và chạy hệ thống từ đầu

```
1  # 1. Clone repository
2  git clone https://github.com/phamminhhoang1985/
   DDM501_Lab1_25MS13285_PhamMinhHoang.git
3  cd DDM501_Lab1_25MS13285_PhamMinhHoang/ml-monitoring
4
5  # 2. Tao thu muc artifacts cho MLflow
6  mkdir C:\mlflow
7
8  # 3. Khoi dong services
9  docker compose up -d
10
11 # 4. Cho services khoi dong (khoang 60 giay)
12 Start-Sleep -Seconds 60
13
14 # 5. Cai dat Python dependencies cho scripts
15 pip install -r scripts/requirements.txt
16
17 # 6. Huan luyen mo hinh ban dau
18 python scripts/training.py
19
20 # 7. (Tuy chon) Chay demo drift & auto-retrain
21 python scripts/simulate_drift_and_retrain.py
22
23 # 8. Chay bo test
24 pip install pytest pytest-cov
25 pytest tests/ --cov=src --cov=api -v
```

Chương B

Cấu hình Docker Compose

Listing B.1: Trích đoạn docker-compose.yml (services chính)

```
1  services:
2    postgres:
3      image: postgres:15-alpine
4      environment:
5        POSTGRES_USER: airflow
6        POSTGRES_PASSWORD: airflow
7        POSTGRES_DB: airflow
8      ports: ["5432:5432"]
9
10   mlflow:
11     build: ./mlflow
12     ports: ["5000:5000"]
13     volumes:
14       - C:/mlflow:/mlflow
15     depends_on: [postgres]
16
17   api:
18     build: ./api
19     ports: ["8000:8000"]
20     volumes:
21       - C:/mlflow:/mlflow
22     environment:
23       MLFLOW_TRACKING_URI: http://mlflow:5000
24
25   airflow-webserver:
26     image: ddm501-t03-airflow:2.8.4
27     ports: ["8080:8080"]
28     volumes:
29       - ./airflow/dags:/opt/airflow/dags
30       - C:/mlflow:/mlflow
31     environment:
32       AIRFLOW__CORE__EXECUTOR: LocalExecutor
33       AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://
        airflow:airflow@postgres/airflow
```

Chương C

Cấu trúc Airflow DAG

Listing C.1: ml_pipeline_dag.py – DAG chính

```
1  from airflow import DAG
2  from airflow.operators.python import PythonOperator
3  from datetime import datetime, timedelta
4
5  default_args = {
6      'owner': 'mlops-team',
7      'retries': 1,
8      'retry_delay': timedelta(minutes=5),
9  }
10
11  with DAG(
12      'ml_pipeline',
13      default_args=default_args,
14      schedule_interval=None, # Trigger thu cong
15      start_date=datetime(2024, 1, 1),
16      catchup=False,
17  ) as dag:
18
19      ingest = PythonOperator(
20          task_id='data_ingestion',
21          python_callable=ingest_data,
22      )
23      clean = PythonOperator(
24          task_id='data_cleaning',
25          python_callable=clean_data,
26      )
27      features = PythonOperator(
28          task_id='feature_engineering',
29          python_callable=engineer_features,
30      )
31      train = PythonOperator(
32          task_id='model_training',
33          python_callable=train_model,
34      )
35      evaluate = PythonOperator(
36          task_id='model_evaluation_and_registration',
37          python_callable=evaluate_and_register,
38      )
39
40      ingest >> clean >> features >> train >> evaluate
```